Lenguaje Java

Relación usa-un

En algunas ocasiones en nuestras clases deberemos usar instancias de otras clases.

Esta relación se implementa pasando como parámetro el objeto usado a la clase que lo utiliza.

```
Hotel unHotel = new Hotel();
unEmpleado.reservarPlaza(unHotel);
```

Se utiliza esta relación asimismo, cuando un método en una clase usa los servicios de otra. Por ej.:

```
public class Empleado {
//........
 public boolean esApto(Medico unMedico)
 {
    return unMedico.check(this);
 }
```

Lenguaje Java

Relación crea-un.

En OO usualmente instanciamos los objetos en main() y los hacemos interactuar.

A veces se necesita que un objeto cree una instancia de otra clase.

```
public class Admision {
    //............
    Paciente unPaciente = new Paciente();
    //.........
 }
```

Lenguaje Java

Reuso de Clases

Se reúsa código cuando al crear nuevas clases, se utilizan clases existentes.

Hay dos formas de trabajar que alientan el reuso:

1) **Composición:** Se crean objetos de clases existentes dentro de la nueva clase. Simplemente se está reusando la funcionalidad del código.

2) **Herencia:** Se crea una nueva clase como un subtipo de una clase existente.

Lenguaje Java

Sintaxis de la Herencia.

En OO, la herencia es una relación de tipo es-un

En realidad en Java siempre se utiliza la herencia, dado que implícitamente se hereda de la clase Object standard de Java.

Para decir que una clase es del tipo de otra, se utiliza la palabra extends seguida del nombre de la clase base. Al hacer esto se heredan los datos y métodos de la clase base.

```
public class Gerente extends Empleado {
    //..........
 }
```

Lenguaje Java

Sintaxis de la composición.

En OO se la conoce como la relación de tipo es-parte-de.

Simplemente se colocan referencias a objetos dentro de la nueva clase.

```
public class Empleado {
    private TarjetaCredito tarjetaC;
    private Domicilio direccion;
    //...........
 }
```

Lenguaje Java

Un ejemplo de Herencia:

```
class CuentaBancaria
 {
    private double saldo;
    public double balance()
    {
       return saldo;
    }
    // otros metodos de CuentaBancaria
 }

 class CuentaCorriente extends CuentaBancaria
 {
    public void escribirCheque(double cantidad)
    { /* código del metodo*/  }
 // otros métodos exclusivos de las cuentas
  // corrientes
 }
```

CuentaCorriente **es una** subclase de CuentaBancaria
CuentaCorriente tiene relación **es-una** con CuentaBancaria.

Lenguaje Java

Ahora es lícito lo siguiente:

```
class MiClase
 {
    public static void unaFuncion()
    {
       CuentaCorriente cc = new CuentaCorriente();
       cc.deposito(100.00);
       System.out.println("Saldo de la cuenta corriente = " +
                cc.balance());
    }
 }
```

Además si una función espera como argumento suyo un objeto CuentaBancaria, podemos pasarle como argumento un objeto CuentaCorriente, ya que es-una CuentaBancaria.

Lenguaje Java

```
public MiClase
 {
    public static void unaFuncion(CuentaBancaria cb)
    {
       // lo que tenga que hacer.....
    }
    public static void otraFuncion()
    {
       CuentaCorriente cc = new CuentaCorriente();
       unaFuncion(cc); // está permitido
    }
 }
```

Se puede pensar en la herencia entonces, como reuso de la interface.

Es posible también tomar un método que ha sido definido en la clase base y modificarlo. Incluso se puede querer utilizar el método de la clase base dentro de la nueva versión.

Lenguaje Java

Veamos el ejemplo de sobreescribir el método Interes() en la subclase CuentaCorriente

```
class CuentaCorriente extends CuentaBancaria
 {
 public void interes()  // cargo el interes mensual
 {
 //las cuentas corrientes tienen un mínimo de 500$ antes de
// cargar el interés
        if (balance() >= 500.0){
          //se calcula el saldo igual que antes
        }
    }
     // métodos de CuentaCorriente
 }
```

Ahora la clase CuentaCorriente hereda todos los métodos de CuentaBancaria, pero para interes() tiene su propia definición.
Para referirse a la superclase desde la clase actual, se utiliza la palabra clave **super.**
Por ej. si en la clase CuentaCorriente queremos referirnos al método Interes de la clase base, desde la subclase, se lo invoca como **super.interes()**

Lenguaje Java

Supongamos que queremos usar los métodos interes() definidos en ambas clases. Por ej:

```
public MiClase{
    {
        public static void unaFuncion(CuentaBancaria
cb)
    {
        // acumulo intereses
        cb.interes();
    }
    public static void otraFuncion()
    {
        CuentaCorriente cc = new CuentaCorriente();
        unaFuncion(cc);  // está permitido

        CuentaBancaria c = new CuentaBancaria();
        unaFuncion(c);  // también es correcto
    }
    }
```

Lenguaje Java

Decimos que el tipo declarado de cb es CuentaBancaria, pero el tipo en ejecución es de cb es CuentaCorriente.

El tipo en ejecución se controla en función de cómo se haya llamado al método.

En Java, la llamada se basa en el tipo en ejecución.

Cuando unaFuncion() se invoca con un objeto CuentaCorriente cc, la llamada cb.interes() invoca al método CuentaCorriente.interes( ); cuando unaFuncion() se invoca con el objeto c que es de tipo CuentaBancaria, la llamada cb.interes() invoca a CuentaBancaria.interes( ).

Lenguaje Java

Inicialización de la clase base

Cuando utilizamos herencia existen dos clases: la clase base y la clase derivada.

Cuando se crea un objeto de la clase derivada, éste realmente contiene dentro de sí un subobjeto de la clase base.

Por ello el objeto de la clase base debe estar correctamente inicializado en el momento de crear un objeto de la clase derivada.

La única garantía para que ello suceda, es realizar la inicialización invocando el constructor de la clase base.

Java automáticamente inserta la llamada al constructor de la clase base en el constructor de la clase derivada.

Lenguaje Java

```
public class Arte
{
 Arte()  {  System.out.println("Constructor de Arte");}
}
public class Dibujo extends Arte
{
 Dibujo() {   System.out.println("Constructor de
Dibujo");}
}
public class Caricatura extends Dibujo
{
 public Caricatura()
 {
    System.out.println("Constructor de Caricatura");
 }
 public static void main(String [] args)
 {
    Caricatura x = new Caricatura();
 }
}
```

Lenguaje Java

La salida de este programa muestra el orden de invocación de los constructores:

Constructor de Arte
Constructor de Dibujo
Constructor de Caricatura

Se puede ver que la clase base es inicializada antes de que los constructores de la clase derivada puedan accederla.
Aunque no se escriba un constructor para Caricatura, el compilador brinda un constructor por defecto que invoca al constructor de la clase base.

Lenguaje Java

Constructores con argumentos

Si se quiere invocar a un constructor de la clase base que tiene un argumento, se debe escribir explícitamente la llamada al constructor de la clase base usando la palabra super y la lista de argumentos apropiada.

```
class Juego
{
 Juego(int i)
{
    System.out.println("Constructor de Juego");
 }
}

class JuegoDeTablero extends Juego
{
 JuegoDeTablero(int i)
{
    super (i); //  llamada al constructor de la clase base
    System.out.println("Constructor de JuegoDeTablero");
 }
}
```

Lenguaje Java

```
public class Ajedrez extends JuegoDeTablero
{
 public Ajedrez()
 {
    super(11); // llamada al constructor de la clase base
    System.out.println("Constructor de Ajedrez");
 }
 public static void main(String args)
 {
    Ajedrez x = new Ajedrez();
 }
}
```

La salida será:
Constructor de Juego
Constructor de JuegoDeTablero
Constructor de Ajedrez

La llamada al constructor de la clase base, debe ser lo primero que se hace dentro del constructor de la clase derivada.

Lenguaje Java

```
class Base
 {
    private int datoBase;
    Base(int nb)
    {
       datoBase = nb;
    System.out.println("Me pasaron el argumento " + nb +
"Estoy  en base");
    }
 }

 class Sub extends Base
 {
    private int datoSub;
    Sub(int nb, int ns)
    {
       super(nb); // invoco al constructor de la clase base
       datoSub = ns;
 System.out.println("Me pasaron el argumento " + ns + "Estoy
en Sub");
    }
 }
```

Lenguaje Java

```
public class BaseSub
 {
    public static void main(String[] args)
    {
       System.out.println("Paso los argumentos 10 y 20");
       Sub sub = new Sub(10, 20);
    }
 }
```

La salida será:

Paso los argumentos 10 y 20
Me pasaron el argumento10 Estoy en base
Me pasaron el argumento20 Estoy en Sub

Como Sub también recibe argumentos, coloco en su lista, los argumentos de los dos constructores: el de la clase base y el propio.
La palabra super permite invocar el constructor de la clase base desde la derivada.

Lenguaje Java

Palabra protected

Ahora se puede comprender la palabra **protected.**

Cuando se quiere que algunas cosas queden ocultas del mundo exterior, pero permitir el acceso a los miembros de las clases derivadas, se usa la palabra protected.

En este caso la palabra nos dice que ese método es privado para todo el mundo, pero disponible para las clases herederas y para aquellas que están en el mismo paquete. O sea protected es automáticamente friendly.

Lo que conviene es dejar los datos miembro como privados y permitir el acceso a los herederos a través de métodos protected.

Lenguaje Java

```
class Super
 {
    public void mpublic(){}
    void mfriendly() {}
    protected void mprotected(){}
    private void mprivate() {}
 }
 class SubClase extends Super
 {
    void a() { mpublic();}
    void b() { mfriendly();}
    void c() { mprotected();}
    // void d() { mprivate();} // error
 }
 public class PruAcceso
 {
    Super obj = new Super();
    void e() { obj.mpublic();}
    void f() { obj.mfriendly();}
    void g() { obj.mprotected();}
    // void h() { obj.mprivate();} // error
 }
```

Lenguaje Java

SubClase y PruAcceso tienen acceso a todos los métodos de Super excepto a private().

Las clases de un paquete diferente tienen acceso sólo a los métodos públicos.

```
import paquetePrinc.Super;

 public class PruAccesoOtroPaquete
 {
    Super obj = new Super();
    void e() { obj.mpublic();}
    // void f() { obj.mfriendly();} // error
    // void g() { obj.mprotected();} //error
    // void h() { obj.mprivate();} // error
 }
```

Lenguaje Java

Conversión

Siempre es lícito dar una referencia a una subclase cuando se solicita una referencia a la clase base:

```
Class MiClase
 {
    public static void unaFuncion(ClaseBase cb) {
/*.....*/ }
    public static void otraFuncion()
    {
        SubClase sc = new SubClase();
        unaFuncion(sc); //funciona bien
    }
 }
```

La conversión de una referencia a una subclase, a una referencia a una clase base es automática, dado que están relacionadas por el principio es-una.

La conversión inversa (de clase base a clase derivada) es lícita, pero se precisa una conversión cast explícita.

Lenguaje Java

```
Class MiClase
 {
    public static void unaFuncion(ClaseBase cb)
    {
        SubClase sc = (SubClase) cb;
        //....
    }
    public static void otraFuncion()
    {
        SubClase sc = new SubClase();
        unaFuncion(sc); //funciona bien
    }
 }
```

Puede utilizarse el operador binario instanceof para asegurarse que un objeto es de un determinado tipo.
Este operador devuelve un booleano true si el objeto a la izquierda del operador de asignación, es del mismo tipo que el objeto a la derecha.

Lenguaje Java

Clases y métodos finales

Ya vimos que una variable de clase puede declararse final significando que su valor no puede cambiarse.

Una clase puede declararse final significando que no puede extenderse.

Un método puede declararse final significando que no puede sobreescribirse en la subclase.

Como no es accesible desde una subclase, un método private siempre es final.

Lenguaje Java

Inicialización y carga de clases

```
class Insecto
 {
    int i= 9;
    int j;
    Insecto() {
       prt("i= "+ i + ", j ="+ j);
       j= 39;
    }
    static int x1 = prt("static Insect.x1
incializada");
    static int prt(String s){
       System.out.println(s);
       return 47;
    }
 }
```

Lenguaje Java

```
public class Escarabajo extends Insecto
 {
    int k = prt("Escarabajo.k inicializada");
    public Escarabajo()
    {
       prt("k= "+ k);
       prt("j= "+j);
    }
    static int x2 = prt("static Escarabajo.x2
incializada");
    public static void main(String[] args)
    {
       prt("Constructor de escarabajos");
       Escarabajo b = new Escarabajo();
    }
 }
```

Lenguaje Java

La salida será:
static Insect.x1 incializada
static Escarabajo.x2 incializada
Constructor de escarabajos
i= 9, j =0
Escarabajo.k inicializada
k= 47
j= 39

En Java el código compilado de cada clase existe en su propio archivo separado. Ese archivo no se carga hasta que el código es necesario.
Se puede decir que “El código de la clase se carga en el momento de su primera utilización”. Esto usualmente no ocurre hasta que se crea el primer objeto, pero la carga también se realiza cuando se accede al primer método o variable estática.

Lenguaje Java

El punto de primer uso también sucede cuando la inicialización estática toma lugar. Todos los objetos estáticos y bloques de código estáticos, serán inicializados en orden textual, en el punto de carga.

Lenguaje Java

En el ejemplo, el orden es el siguiente:

1. Cuando se ejecuta, se trata de acceder a Escarabajo.main() (Un método estático). El cargador encuentra el código compilado para la clase Escarabajo (en un archivo Escarabajo.class)
2. En este proceso, se encuentra que Escarabajo tiene una clase base, a la cual carga. Si esta a su vez tiene una clase base, se carga y así...
3. Luego se ejecuta la inicialización estática en la clase base raíz (Insecto), luego en la clase derivada y así...
4. Ya se han cargado las clases, ahora puede crearse el objeto.
5. Las primitivas en el objeto, se setean a su valor default y las referencias a objetos a null.
6. Se llama el constructor de la clase base (en este caso es automático, se puede forzar usando super).
7. Luego de que el constructor de la clase base finaliza la tarea, las variables de instancia son inicializadas en el orden textual.
8. Finalmente se ejecuta el resto del bloque del constructor.

Lenguaje Java

Polimorfismo

Provee otra separación entre la interface y la implementación. Permite mejorar la organización del código así como la creación de programas extensibles que “crezcan” durante el desarrollo de un proyecto, cuando se necesiten nuevas características.

Hemos visto que el mecanismo de la herencia permite tratar un objeto como de su propio tipo o como del tipo de su clase base. Esta habilidad permite a muchos tipos ser tratados como si fueran uno solo y una sóla pieza de código trabajar con todos ellos igualmente.

La llamada al método polimórfico permite a un tipo expresar su distinción de otro (tipo similar) mientras ambos sean derivados de la misma clase base.

Al polimorfismo se lo asocia con el témino binding dinámico o late binding.

Lenguaje Java

```
class Nota
 {
    private int value;
    private Nota(int val) { value = val;}
    public static final Nota
       MID = new Nota(0),
       C_SHARP = new Nota(1),
       B_FLAT = new Nota(2);
 }
 class Instrumento
 {
    public void play(Nota n)
    {
       System.out.println("Instrument.play()");
    }
 }
```

Lenguaje Java

```
class Viento extends Instrumento
 {
    public void play(Nota n) {
       System.out.println("Viento.play()");
    }
 }
 public class Musica
 {
    public static void tono(Instrumento i)
    {
       i.play(Nota.MID);
       //...
    }
    public static void main(String[] args)
    {
       Viento flauta = new Viento();
       tono(flauta); // Upcasting
    }
 }
```

Lenguaje Java

Tratar una referencia a un objeto como si fuera de su tipo base, es lo que se llama Upcasting.
El método Music.tono acepta una referencia a un Instrumento y también de cualquier cosa derivada de Instrumento.
En main() se puede observar como una referencia a Viento se pasa a tono() sin necesidad de casting.
Si en el método tono, en lugar de la referencia a un Instrumento como argumento, se hubiera puesto la referencia a un Viento, habría que escribir un nuevo método tono para cada tipo de instrumento.

En el ejemplo anterior, si se agregaran clases de más tipos de instrumentos: cuerda y percusión y en el método tono() no se aceptara una referencia a la clase base como argumento, habría que escribir tres métodos tono() diferentes, uno que reciba Viento como argumento, otro Cuerda y otro Percusión.

El polimorfismo permite olvidar la existencia de clases derivadas y escribir el código que trate sólo con la clase base.

Lenguaje Java

Llamada a método – binding

La conexión entre la llamada a un método y el cuerpo del método se llama binding.

***Early binding*:** el binding se ejecuta entes de que se corra el programa, por el compilador y el linkeador.
***Late binding*:** Significa que el binding se realiza en tiempo de ejcución, basado en el tipo del objeto. También se llama dynamic binding o run-time binding.

Todos los métodos en Java usan late binding, a menos que hayan sido declarados como final. Esto previene la posibilidad de sobreescribir el método.
La forma correcta de usar el polimorfismo en la programación OO, es escribir el código para la clase base, sabiendo que todas las clases derivadas trabajarán correctamente usando el mismo código.

Lenguaje Java

El ejemplo clásico es Figura.

Figura
dibujar()
borrar()

Círculo
dibujar()
borrar()

Rectángulo
dibujar()
borrar()

Triángulo
dibujar()
borrar()

Lenguaje Java

El upcasting puede verse en una declaración como ésta:

```
Figura s = new Circulo();
```

La referencia resultante se asigna a una Figura. Si ahora se invoca uno de los métodos de la clase base:

```
s.dibujar();
```

el método que se ejecuta es el correspondiente a Círculo y no a su clase base, debido al late binding y polimorfismo.

La clase Figura establece la interface común a cualquiera heredada de Figura, o sea todas sus derivadas pueden ser dibujadas y borradas.

Las clases derivadas override (sobreescriben) esas definiciones para proveer comportamiento único para cada tipo específico de figura.

Lenguaje Java

Overriding vs. Overloading (Sobreescritura vs. Sobrecarga)

En el siguiente ejemplo, la interface del método play() se cambia en el proceso de overriding; en lugar de overrinding, sucede overloading. El compilador no dará mensaje de error, ya que esto es permitido, pero el comportamiento, probablemente no será el esperado.

```
class NotaX
 {
    public static final int
       MID = 0,
       C_SHARP = 1,
       B_FLAT = 2;
 }
```

Lenguaje Java

```
class InstrumentoX
 {
    public void play(int NotaX) // NotaX aquí se usa como
//identificador
    {
       System.out.println("InstrumentoX.play()");

 }
 }
 class Wind extends InstrumentoX
 {
    // cambia la interface del método
    public void play(NotaX n) {
       System.out.println("Wind.play(NotaX n)");
    }
 }
```

Aquí el programador trató de hacer override, pero sobrecargó.

Lenguaje Java

Clases abstractas

En los ejemplos se vio claramente que la intención de la clase Instrumento, es crear una interface común para todas las clases derivadas de ella.
La única razón para crear esta interface es que después pueda ser expresada en forma diferente para las distintas subclases.
Se crea una clase abstracta, como puede ser Instrumento, cuando se quiere manipular un conjunto de clases a través de su interface común. Los métodos derivados que coincidan con la signatura de la declaración en la clase base, serán invocados usando el mecanismo de binding dinámico.
No tiene sentido crear instancias de las clases abstractas. Para evitarlo, Java provee un mecanismo llamado método abstracto, el cual tiene una declaración, pero no tiene cuerpo.

```
abstract void f();
```

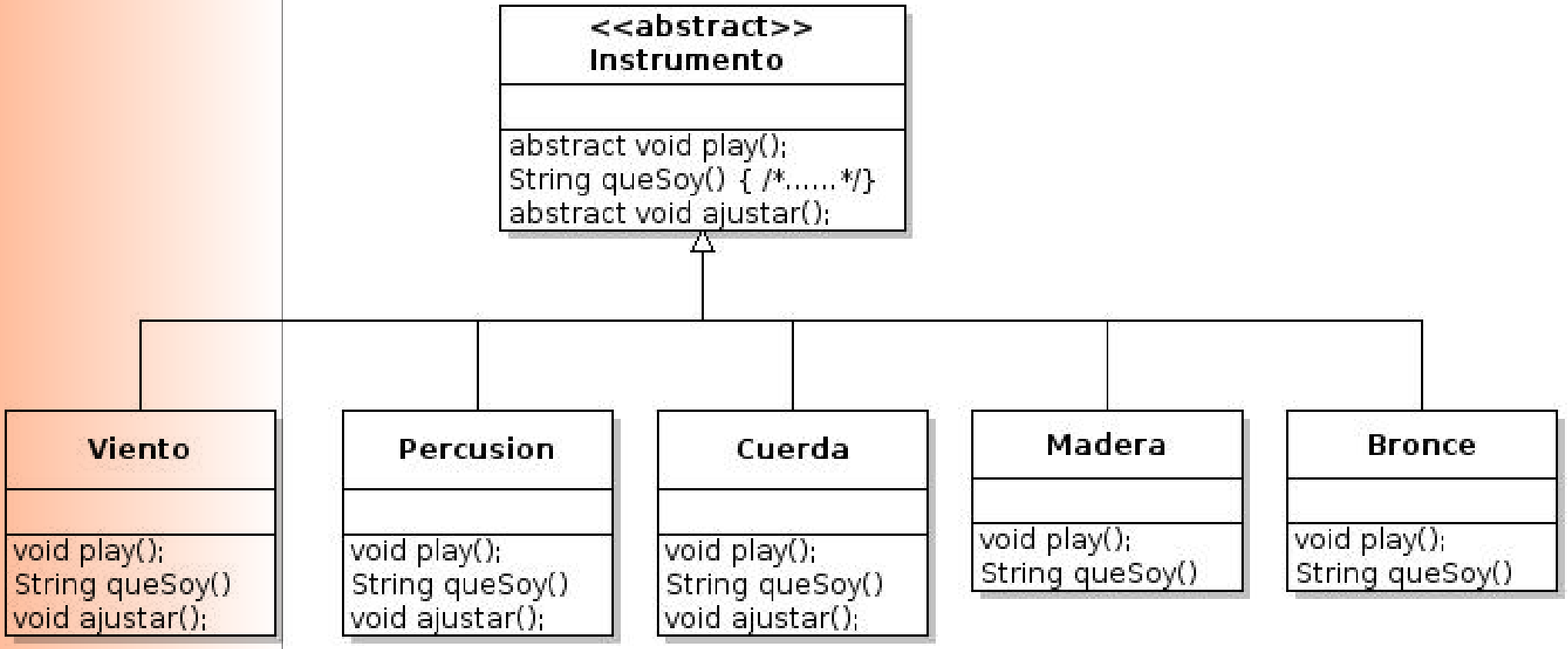
Una clase que contiene uno o más métodos abstractos, se llama clase abstracta y debe ser calificada con **abstract.**

Lenguaje Java

Si se hereda de una clase abstracta y se construyen objetos del nuevo tipo, se deben proveer definiciones de métodos para todos los métodos abstactos de la clase base.
Si esto no sucede, la clase derivada también será abstracta y el compilador lo forzará a calificar esa clase con la palabra abstract.

Es posible crear una clase abstracta sin incluir ningún método abstract. Esto es útil cuando se tiene una clase en la cual no tiene sentido tener métodos abstractos.
Es correcto crear clases y métodos abstractos, ya que ellos hacen explícita la abstracción que provee una clase.

Lenguaje Java



Lenguaje Java

Constructores y polimorfismo – Orden de invocación.

```
class Comida {
        Comida() { System.out.println("Comida()"); }
    }
    class Pan {
        Pan()   { System.out.println("Pan()");    }
    }
    class Queso {
        Queso() { System.out.println("Queso()");}
    }
    class Jamon {
        Jamon() { System.out.println("Jamon()"); }
    }
    class Almuerzo extends Comida {
        Almuerzo() { System.out.println("Almuerzo()");}
    }
    class AlmuerzoPortable extends Almuerzo {
        AlmuerzoPortable() { System.out.println("AlmuerzoPortable()");}
    }
```

Lenguaje Java

```
public class Sandwich extends AlmuerzoPortable {
        Pan p = new Pan(); // composición
        Queso q = new Queso(); //composición
        Jamon j = new Jamon(); // composición
        public Sandwich() { System.out.println("Sandwich()"); }
        public static void main(String[] args)
        {
            new Sandwich();
        }
    }
```

Cuando se crea un objeto Sandwich desde main(), la salida será:

Comida()
Almuerzo()
AlmuerzoPortable()
Pan()
Queso()
Jamon()
Sandwich()

Lenguaje Java

Esto significa que el orden de llamada de los constructores para un objeto compuesto, es el que sigue:

1. Se llama el constructor de la clase base. Se sigue este paso según la jerarquía hasta que se alcanza la última derivada.
2. Los inicializadores de objetos miembros se llaman en su mismo orden de declaración.
3. Se llama el cuerpo del constructor de la clase derivada.

Lenguaje Java

Herencia y finalize().

Cuando se utiliza herencia, se debe sobreescribir finalize() en la clase derivada, si se tiene que realizar algún tipo de cleanup, que deba suceder como parte de garbage collection.

Cuando se sobreescribe finalize() en la clase derivada, es importante recordar que se debe llamar a la versión de finalize () de la clase base, dado que de otro modo no sucederá la finalización de la clase base.